

Reward-tilting Problems

Jaemoo Choi

- **Lecture 3** Reward-tilting: sampling from $p_1^* \propto p_1^{\text{base}} e^{\beta r}$ with a frozen pretrained model. Test-time scaling (the tilting identity, the weight process, guidance for inverse problems), cross entropy (weighted denoising score matching, its proximal variant), relative entropy (the stochastic control problem and adjoint matching), and stochastic density control (sampling a ratio of two pretrained models).
- **Lecture 4** The **Bridge Matching Sampler**: sampling from an energy with no pretrained model to tilt — the interpolant’s velocity and score, both least squares whose label is the force read at a self-generated endpoint, and JANUS.

Date: August 31, 2026

1 The Reward-Tilting Problem

Notations μ : the prior of the pretrained model at $t = 0$ (Gaussian noise); p^{base} : the law of the pretrained generative process on $[0, 1]$, with marginals p_t^{base} , transitions $p_{t|s}^{\text{base}}$ and $p_{t|1}^{\text{base}}$ (the noising kernel); $r : \mathbb{R}^d \rightarrow \mathbb{R}$: the reward, ∇r available; $\beta > 0$: the inverse temperature; W_t : the Brownian motion driving whichever process is being simulated; $u_t(x)$: a control, always σ -scaled (it enters the drift as $\sigma_t u_t$); F : a test function. Time runs from noise ($t = 0$) to data ($t = 1$).

Setting: the pretrained model Given is a diffusion model: a score network $\nabla \log p_t^{\text{base}}$ and the noising pair (f_t, σ_t) . Its generative process and the equation of its marginals are

$$p^{\text{base}} : \quad dX_t = b_t(X_t) dt + \sigma_t dW_t, \quad X_0 \sim \mu, \quad b_t := -f_t + \sigma_t^2 \nabla \log p_t^{\text{base}}, \quad (1)$$

$$\partial_t p_t^{\text{base}} = \nabla \cdot (f_t p_t^{\text{base}}) - \frac{\sigma_t^2}{2} \Delta p_t^{\text{base}} \quad (\text{the noising equation, read backward}). \quad (2)$$

(1) is Anderson’s reversal of the noising dynamics (Anderson, 1982; Song et al., 2021); (2) is its Fokker–Planck equation.

Convention In the lecture the sampler is allowed a free reference process p^b , and the pretrained model is only one choice of it. *Throughout these notes* $p^b = p^{\text{base}}$: we simulate on top of the pretrained drift b_t . This is the case worth remembering — every formula below loses a term.

Definition 1.1 (The reward-tilted target).

$$p_1^*(x) \propto p_1^{\text{base}}(x) e^{\beta r(x)}; \quad \text{on path space} \quad \frac{dp^*}{dp^{\text{base}}}(X) = \frac{e^{\beta r(X_1)}}{Z}, \quad Z = \mathbb{E}_{p^{\text{base}}} [e^{\beta r(X_1)}]. \quad (3)$$

Remark (The lift is consistent, and it is free). The path-space tilt depends on X_1 only, so it does not touch the conditional law of the path given its endpoint: $p_{\cdot|1}^* = p_{\cdot|1}^{\text{base}}$, and the time-1 marginal of the path measure (3) is exactly the tilted density. Consequently

$$p_t^* = \int p_{t|1}^{\text{base}}(\cdot | x_1) p_1^*(x_1) dx_1 \quad (\text{a mixture of noising kernels}). \quad (4)$$

The network stays frozen. Two levers remain: the SDE we simulate, and a weight we carry.

The proposal For a control u , the proposal p^u is

$$p^u : \quad dX_t = (b_t(X_t) + \sigma_t u_t(X_t)) dt + \sigma_t dW_t, \quad X_0 \sim \mu. \quad (5)$$

Theorem 1.2 (Girsanov). *Along trajectories sampled from p^u , with W the Brownian motion of (5),*

$$\log \frac{dp^u}{dp^{\text{base}}}(X) = \int_0^1 u_t(X_t)^\top dW_t + \int_0^1 \frac{1}{2} \|u_t(X_t)\|^2 dt, \quad D_{\text{KL}}(p^u \| p^{\text{base}}) = \mathbb{E}_{p^u} \left[\int_0^1 \frac{1}{2} \|u_t\|^2 dt \right]. \quad (6)$$

Proof. The law-free form is $\log \frac{dp^u}{dp^{\text{base}}}(X) = \int_0^1 u_t^\top \sigma_t^{-1} (dX_t - b_t dt) - \int_0^1 \frac{1}{2} \|u_t\|^2 dt$ (the shared drift b cancels). Sampled from p^u , $\sigma_t^{-1} (dX_t - b_t dt) = u_t dt + dW_t$, which flips the half to $+$. The stochastic integral is a p^u -martingale, hence mean zero: the KL identity. Sampled from p^{base} instead, $\sigma_t^{-1} (dX_t - b_t dt) = dW_t$ and the half keeps its minus — one formula, two signs; the sampled dynamics decides. $\square$

2 General Test-Time Scaling

2.1 The Tilting Identity

Definition 2.1 (The interpolated reward). Choose any smooth r_t with $r_0 \equiv 0$ and $r_1 = \beta r$ — the reward credited by time t , so that the reference tilted by e^{r_t} starts at the reference and ends at the target:

$$r_0 \equiv 0, \quad r_1 = \beta r, \quad dp^* \propto e^{r_1(X_1)} dp^{\text{base}}. \quad (7)$$

Theorem 2.2 (The tilting identity). *Put $U_t := \sigma_t \nabla r_t$. Then for every control u , along trajectories of p^u ,*

$$\begin{aligned} \log \frac{dp^*}{dp^u}(X) &= \int_0^1 \left(\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} (\Delta r_t + \|\nabla r_t\|^2) - \frac{1}{2} \|u_t - U_t\|^2 \right) (X_t) dt \\ &\quad + \int_0^1 (U_t - u_t)(X_t)^\top dW_t - \log Z. \end{aligned} \quad (8)$$

Proof. Three steps.

(i) *Chain rule.* $\log \frac{dp^*}{dp^u} = r_1(X_1) + \log \frac{dp^{\text{base}}}{dp^u}(X) - \log Z$, and by [theorem 1.2](#) the middle term is $-\int_0^1 u_t^\top dW_t - \int_0^1 \frac{1}{2} \|u_t\|^2 dt$.

(ii) *Itô.* Since $r_0 \equiv 0$, $r_1(X_1) = \int_0^1 dr_t(X_t)$ — no boundary term. Along (5),

$$dr_t(X_t) = \left[(\partial_t + b_t \cdot \nabla + \frac{\sigma_t^2}{2} \Delta) r_t + u_t^\top U_t \right] (X_t) dt + U_t(X_t)^\top dW_t, \quad (9)$$

using $\sigma_t u_t \cdot \nabla r_t = u_t^\top U_t$.

(iii) *Complete the square.* Adding (i) and (ii), the dW -terms give $U_t - u_t$ and the dt -terms give $(\partial_t + b_t \cdot \nabla + \frac{\sigma_t^2}{2} \Delta) r_t + u_t^\top U_t - \frac{1}{2} \|u_t\|^2$; with $u^\top U - \frac{1}{2} \|u\|^2 = \frac{1}{2} \|U\|^2 - \frac{1}{2} \|u - U\|^2$ and $\frac{1}{2} \|U_t\|^2 = \frac{\sigma_t^2}{2} \|\nabla r_t\|^2$ this is the integrand of (8). $\square$

Remark (The control-free part of the weight). The first three terms of (8) are the only place the choice of r_t is felt. By $\Delta e^r = e^r (\Delta r + \|\nabla r\|^2)$ they are a ratio,

$$\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} (\Delta r_t + \|\nabla r_t\|^2) = \frac{(\partial_t + b_t \cdot \nabla + \frac{\sigma_t^2}{2} \Delta) e^{r_t}}{e^{r_t}}, \quad (10)$$

which vanishes identically if and only if $e^{r_t(X_t)}$ is a p^{base} -martingale, that is, if and only if $e^{r_t} = \mathbb{E}_{p^{\text{base}}} [e^{\beta r(X_1)} | X_t]$. So it is the martingale (Hamilton–Jacobi–Bellman) residual of the interpolation we picked by hand; it is explicit for every explicit r_t . Tractability is never the issue — weight variance is.

Remark (The meaning of U_t). $U_t = \sigma_t \nabla r_t$ steers up the tilt; it is Lecture 2’s optimal control $u_t^* = \sigma_t \nabla \log \varphi_t$ with e^{r_t} in place of φ_t . At $u = U$ the identity collapses to

$$dp^* \propto \exp \left(\int_0^1 \left(\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} (\Delta r_t + \|\nabla r_t\|^2) \right) (X_t) dt \right) dp^U.$$

Steering kills the martingale part of the weight, never the drift. Both vanish exactly for the value function of (10), where $p^U = p^*$ and the weights are constant.

2.2 The Weight Process and the Sampler

Corollary 2.3 (The weight process). *Simulate the pair $(X_t, \log w_t)$ from $X_0 \sim \mu$, $w_0 = 1$, driven by the same W_t :*

$$\begin{aligned} dX_t &= (b_t + \sigma_t u_t)(X_t) dt + \sigma_t dW_t, \\ d \log w_t &= \left(\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} (\Delta r_t + \|\nabla r_t\|^2) - \frac{1}{2} \|u_t - U_t\|^2 \right) (X_t) dt \\ &\quad + (U_t - u_t)(X_t)^\top dW_t. \end{aligned} \tag{11}$$

Then $w_1 = Z \frac{dp^*}{dp^u}(X)$, and for every test function F ,

$$\frac{\mathbb{E}_{p^u} [w_1 F(X_1)]}{\mathbb{E}_{p^u} [w_1]} = \mathbb{E}_{p_1^*} [F]. \tag{12}$$

Proof. Integrating (11) from 0 to 1 gives $\log w_1 = \log \frac{dp^*}{dp^u}(X) + \log Z$ by theorem 2.2. Hence $\mathbb{E}_{p^u} [w_1 F(X_1)] = Z \mathbb{E}_{p^*} [F(X_1)]$, and $F \equiv 1$ gives $\mathbb{E}_{p^u} [w_1] = Z$: the unknown normalizer cancels in the ratio. $\square$

Algorithm 1 Test-time scaling: the weighted sampler (Wu et al., 2023; Singhal et al., 2025; Skreta et al., 2025)

- 1: Choose an interpolation r_t and a control u ; form $U_t = \sigma_t \nabla r_t$.
 - 2: Draw N particles $X_0^i \sim \mu$, set $\log w_0^i = 0$.
 - 3: Euler step the state with $\Delta W_t^i \sim \mathcal{N}(0, \Delta t I)$: $X_{t+\Delta t}^i = X_t^i + (b_t + \sigma_t u_t)(X_t^i) \Delta t + \sigma_t \Delta W_t^i$.
 - 4: Euler step its log-weight with the *same* increment: $\log w_{t+\Delta t}^i = \log w_t^i + \left(\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} (\Delta r_t + \|\nabla r_t\|^2) - \frac{1}{2} \|u_t - U_t\|^2 \right) (X_t^i) \Delta t + (U_t - u_t)(X_t^i)^\top \Delta W_t^i$.
 - 5: Optionally resample the particles by their weights and reset them.
 - 6: Report $\sum_i w_1^i F(X_1^i) / \sum_i w_1^i$, or the weighted particles themselves.
-

Three interpolated rewards Any r_t with $r_0 \equiv 0$, $r_1 = \beta r$ is admissible; three are standard:

$$r_t(x) = \beta_t r(x) \quad (\text{schedule}), \quad r_t(x) = \beta_t r(\hat{X}_1(x)), \quad \hat{X}_1(x) = \mathbb{E}_{p^{\text{base}}} [X_1 \mid X_t = x] \quad (\text{denoised prediction}), \tag{13}$$

$$r_t(x) = \log \mathbb{E}_{p^{\text{base}}} [e^{\beta r(X_1)} \mid X_t = x] \quad (\text{the value function: (10) vanishes, constant weights, } p^U = p^*).$$

Remark (Advantage and limitation). No training: the network stays frozen and the whole cost is inference-time simulation — any reward, any β , immediately. But everything rests on the proposal. If p^u places little mass where p_1^* lives, the weights degenerate and the effective sample size $N_{\text{eff}} = 1 / \sum_i (w^i)^2$ collapses. A poor proposal is paid in weight variance; steering toward U_t is the remedy.

2.3 Example: Guidance for Inverse Problems

Example 2.4 (The posterior is a tilt). A measurement $y = \mathcal{A}(X_1) + \eta$ is observed, with a known forward operator $\mathcal{A}$ and noise $\eta \sim \mathcal{N}(0, \sigma_y^2 I)$; the prior is the pretrained model. Take the measurement log-likelihood as the reward:

$$r(x) := \log p(y | x) = -\frac{\|y - \mathcal{A}(x)\|^2}{2\sigma_y^2} + \text{const}, \quad \beta = 1 \quad \implies \quad p_1^*(x) \propto p_1^{\text{base}}(x) p(y | x) = p(x | y). \quad (14)$$

The tilted target (3) is the Bayesian posterior and $Z = p(y)$ is the evidence. Nothing is trained and nothing is learned: r and ∇r are closed form, which is why inverse problems are the standard testbed for every method of this section.

The exact guidance Take the value function of (13) as the interpolation. At $\beta = 1$ it is the *noisy likelihood*, and its gradient is the classical guidance term (Dhariwal and Nichol, 2021):

$$e^{r_t(x)} = \mathbb{E}_{p^{\text{base}}} [p(y | X_1) | X_t = x] =: p_t(y | x), \quad U_t(x) = \sigma_t \nabla \log p_t(y | x). \quad (15)$$

This is the value function, so the residual (10) vanishes and steering alone — no weight — samples the posterior: the tilted drift splits into the pretrained score plus guidance, $-f_t + \sigma_t^2 (\nabla \log p_t^{\text{base}} + \nabla \log p_t(y | \cdot))$. But $p_t(y | x)$ averages the likelihood over the denoising posterior $p_{1|t}^{\text{base}}(\cdot | x)$, the model’s own law of the clean sample given the noisy one, and that average is not available. Every method below puts a computable r_t in its place. The substitution is always legal — definition 2.1 constrains only the endpoints — and (10) prices it: the residual no longer vanishes.

The one-step estimate For linear f_t the noising kernel is Gaussian, $p_{t|1}^{\text{base}}(\cdot | x_1) = \mathcal{N}(\kappa_t x_1, s_t^2 I)$ with $\kappa_1 = 1$, $s_1 = 0$, and Tweedie’s formula (Efron, 2011) rearranges the score network into a denoiser at no cost:

$$\hat{X}_1(x) = \mathbb{E}_{p^{\text{base}}} [X_1 | X_t = x] = \frac{x + s_t^2 \nabla \log p_t^{\text{base}}(x)}{\kappa_t} \quad (\text{one network call}). \quad (16)$$

Diffusion posterior sampling (Chung et al., 2023) evaluates the likelihood *at* that average instead of averaging the likelihood — the denoised-prediction interpolation of (13):

$$r_t(x) = \beta_t \log p(y | \hat{X}_1(x)) = -\beta_t \frac{\|y - \mathcal{A}(\hat{X}_1(x))\|^2}{2\sigma_y^2}, \quad \nabla r_t(x) = \beta_t \left(\frac{\partial \hat{X}_1}{\partial x}(x) \right)^\top \nabla \log p(y | \hat{X}_1(x)). \quad (17)$$

It is admissible: $\hat{X}_1(x) \rightarrow x$ as $t \rightarrow 1$, so $r_1 = \beta r$ once $\beta_1 = \beta$, and the schedule supplies $r_0 \equiv 0$. The gradient is one vector–Jacobian product through the frozen network: one backward pass per step, and no training.

Proposition 2.5 (What the one-step estimate keeps and what it drops). *Write $\Sigma_t(x) := \text{Cov}_{p^{\text{base}}} [X_1 | X_t = x]$. Then, exactly,*

$$\Sigma_t(x) = \frac{s_t^2}{\kappa_t} \frac{\partial \hat{X}_1}{\partial x}(x), \quad (18)$$

and if $\mathcal{A}$ is linear and the denoising posterior is Gaussian, the exact interpolation (15) is

$$\begin{aligned} r_t(x) = & -\frac{1}{2} (y - \mathcal{A}\hat{X}_1(x))^\top (\sigma_y^2 I + \mathcal{A} \Sigma_t(x) \mathcal{A}^\top)^{-1} (y - \mathcal{A}\hat{X}_1(x)) \\ & -\frac{1}{2} \log \det (\sigma_y^2 I + \mathcal{A} \Sigma_t(x) \mathcal{A}^\top) + \text{const}. \end{aligned} \quad (19)$$

The one-step estimate (17) is (19) with $\Sigma_t \equiv 0$: the right mean, no covariance.

Proof. (i) *The covariance.* Bayes gives $p_{1|t}^{\text{base}}(x_1 | x) \propto p_1^{\text{base}}(x_1) \mathcal{N}(x; \kappa_t x_1, s_t^2 I)$, so $\nabla_x \log p_{1|t}^{\text{base}}(x_1 | x) = (\kappa_t x_1 - x)/s_t^2 - \nabla \log p_t^{\text{base}}(x)$, the subtracted term being the conditional mean of the first — that is (16).

Differentiating $\hat{X}_1(x) = \int x_1 p_{1|t}^{\text{base}}(x_1 | x) dx_1$ under the integral therefore leaves a centered second moment, $\partial_x \hat{X}_1(x) = \frac{\kappa_t}{s_t^2} \text{Cov}_{p^{\text{base}}}[X_1 | X_t = x]$.

(ii) *The convolution.* A linear image of a Gaussian plus independent Gaussian noise is Gaussian: when $p_{1|t}^{\text{base}}(\cdot | x) = \mathcal{N}(\hat{X}_1(x), \Sigma_t(x))$, $p_t(y | x) = \int \mathcal{N}(y; \mathcal{A}x_1, \sigma_y^2 I) p_{1|t}^{\text{base}}(x_1 | x) dx_1 = \mathcal{N}(y; \mathcal{A}\hat{X}_1(x), \sigma_y^2 I + \mathcal{A}\Sigma_t(x)\mathcal{A}^\top)$. Take logarithms. $\square$

method	the interpolated reward $r_t(x)$	weight
DPS (Chung et al., 2023)	$\beta_t \log p(y \hat{X}_1(x))$, (17)	dropped
PIGDM (Song et al., 2023)	(19) with a hand-set $\Sigma_t = \lambda_t^2 I$	dropped
second-order Tweedie (Boys et al., 2024; Peng et al., 2024)	(19) with Σ_t of (18)	dropped
DDRM, DDNM (Kawar et al., 2022; Wang et al., 2023)	$\hat{X}_1$ projected onto $\{\mathcal{A}x_1 = y\}$ — not a tilt	—
TDS (Wu et al., 2023), FK steering (Singhal et al., 2025)	any row above, used as the twist	carried

Table 1 Guidance for inverse problems. Each method is a choice of r_t in [definition 2.1](#), simulated with $u = U_t = \sigma_t \nabla r_t$; they differ in how much of the denoising posterior $p_{1|t}^{\text{base}}$ they keep, and in whether the weight of [corollary 2.3](#) is carried.

Remark (Guidance chooses r_t ; the weight is what makes it exact). Every row of [table 1](#) is admissible, so [theorem 2.2](#) holds for each of them: the whole deficiency of an approximation sits in the residual (10) and in the martingale term. Steering without a weight samples p^U , which by (8) at $u = U$ misses p^* by the exponential of the time integral of that residual — zero only for the value function (15). Carrying the weight and resampling restores exactness as $N \rightarrow \infty$, for any of these twists (Wu et al., 2023; Singhal et al., 2025). Two readings for practice: the tuned step size that guidance methods put in front of the likelihood gradient is the schedule β_t , a knob on the proposal and never on the target; and any part of r_t free of x — the log det of (19) for an x -free Σ_t , the normalizer of the likelihood — shifts all particles equally and cancels in the self-normalization.

Remark (No Laplacian is ever formed). The residual (10) contains Δr_t , a Laplacian through the network for the choice (17), and no implementation computes it. By step (i) of the proof of [theorem 2.2](#), $\log w_1 = r_1(X_1) + \log \frac{dp^{\text{base}}}{dp^u}(X)$, and on the simulation grid $0 = t_0 < \dots < t_K = 1$ both terms telescope into evaluations:

$$\log w_1 = \sum_{k=0}^{K-1} \left[\underbrace{r_{t_{k+1}}(X_{t_{k+1}}) - r_{t_k}(X_{t_k})}_{\text{successive twists}} + \underbrace{\log \frac{p_{t_{k+1}|t_k}^{\text{base}}(X_{t_{k+1}} | X_{t_k})}{p_{t_{k+1}|t_k}^u(X_{t_{k+1}} | X_{t_k})}}_{\text{two Gaussian transitions}} \right], \quad (20)$$

using $r_0 \equiv 0$. Itô’s formula on the first bracket is what produced the dt and dW_t integrands of [theorem 2.2](#); in code one evaluates the twist twice per step and subtracts. This is the weight of twisted sequential Monte Carlo (Wu et al., 2023).

Nor is (20) an approximation of [corollary 2.3](#) — it is the same weight, cut at the grid instead of infinitesimally. The first bracket telescopes to $r_1(X_1)$ on every grid, path by path, with no error at any Δt . With the exact kernels, the second bracket is the likelihood ratio of the two chains restricted to $(X_{t_0}, \dots, X_{t_K})$, that is, the conditional expectation of the continuous weight given the simulated skeleton — so the self-normalized estimator (12) remains consistent on any finite grid, and the weight’s variance only drops (Rao–Blackwell). In code the kernels are the Euler–Maruyama Gaussians, whose log-ratio is the increment $-u^\top \Delta W - \frac{1}{2} \|u\|^2 \Delta t$; (20) is then still the *exact* likelihood ratio between the two chains actually simulated, so the reweighting itself introduces no bias — what remains is only the $O(\Delta t)$ weak error of simulating the base model, which any sampler of p^{base} carries, weighted or not. Euler-stepping (11) instead would linearize the twist increment — discretization error in the weight itself, on top of forming Δr_t .

Remark (The one-step expansion: what matches what). Over one step $\Delta X = (b_t + \sigma_t u_t) \Delta t + \sigma_t \Delta W$, the

two brackets of (20) expand to

$$\begin{aligned} \log \frac{p_{t+\Delta t|t}^{\text{base}}}{p_{t+\Delta t|t}^u}(X_{t+\Delta t} | X_t) &= -u_t^\top \Delta W - \frac{1}{2} \|u_t\|^2 \Delta t, \\ r_{t+\Delta t}(X_{t+\Delta t}) - r_t(X_t) &= \left[\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} \Delta r_t + u_t^\top U_t \right] \Delta t + U_t^\top \Delta W + o(\Delta t), \end{aligned}$$

steps (i) and (ii) of the proof of [theorem 2.2](#), one grid step at a time; for the two Euler kernels the first line is an algebraic identity, with no remainder. Summing them is step (iii): the noise is $(U_t - u_t)^\top \Delta W$, and the drift completes the square, $u_t^\top U_t - \frac{1}{2} \|u_t\|^2 = \frac{\sigma_t^2}{2} \|\nabla r_t\|^2 - \frac{1}{2} \|u_t - U_t\|^2$, giving the integrand of (11) term for term — the Laplacian and the $+U_t^\top dW_t$ half of the noise live in the twist bracket, the $-u_t^\top dW_t$ half and the $-\frac{1}{2} \|u_t\|^2$ in the transition bracket, and $\frac{\sigma_t^2}{2} \|\nabla r_t\|^2 - \frac{1}{2} \|u_t - U_t\|^2$ is the recombination of the two. (20) sums the left-hand sides as they stand; (11) sums the right-hand sides with the $o(\Delta t)$ dropped — the same weight, before and after the expansion, and the twist row’s Taylor remainder is the only inexact entry.

3 Cross Entropy: Weighted Denoising Score Matching

3.1 The Cross-Entropy Problem

Definition 3.1 (The trainee and the objective). Train a control u^θ so that its law matches the target path measure:

$$\begin{aligned} p^{u^\theta} : \quad dX_t &= (-f_t + \sigma_t u_t^\theta)(X_t) dt + \sigma_t dW_t, \quad X_0 \sim \mu, \\ \min_\theta \text{CE}(p^*, p^{u^\theta}) &:= \min_\theta \mathbb{E}_{p^*} \left[-\log \frac{dp^{u^\theta}}{dp^{\text{base}}}(X) \right]. \end{aligned} \quad (21)$$

Since $D_{\text{KL}}(p^* \| p^{u^\theta}) = \text{CE}(p^*, p^{u^\theta}) + \mathbb{E}_{p^*} \log \frac{dp^*}{dp^{\text{base}}}$ and the second term is θ -free, this is $\min_\theta D_{\text{KL}}(p^* \| p^{u^\theta})$ — the *forward* KL, the unknown first.

Proposition 3.2 (The importance-weighted cross entropy). *Let $\bar{u}$ be any control and $p^{\bar{u}}$ its law (5). Then, for any functional Φ ,*

$$\mathbb{E}_{p^*} [\Phi(X)] = \mathbb{E}_{p^{\bar{u}}} \left[\frac{dp^*}{dp^{\bar{u}}}(X) \Phi(X) \right], \quad \log \frac{dp^*}{dp^{\bar{u}}}(X) = \beta r(X_1) - \int_0^1 \bar{u}_t^\top dW_t - \frac{1}{2} \int_0^1 \|\bar{u}_t\|^2 dt - \log Z. \quad (22)$$

Every term is available during the simulation; $\log Z$ is a constant and disappears in a self-normalized (softmax) weight.

Proof. Change of measure, then split through the base: $\frac{dp^*}{dp^{\bar{u}}} = \frac{dp^*}{dp^{\text{base}}} \cdot \frac{dp^{\text{base}}}{dp^{\bar{u}}}$; the first factor is $e^{\beta r(X_1)}/Z$ by (3), the second is [theorem 1.2](#) with the sign of (6) reversed. (It is (8) for the interpolation $r_t = \beta_t r$ with β_t the indicator of $t = 1$; here the endpoint form is all we need.) $\square$

3.2 Weighted Denoising Score Matching

Theorem 3.3 (Cross entropy equals weighted denoising). *p^* is the diffusion $dX_t = (-f_t + \sigma_t^2 \nabla \log p_t^*)(X_t) dt + \sigma_t dW_t$, and*

$$D_{\text{KL}}(p^* \| p^{u^\theta}) = \frac{1}{2} \mathbb{E}_{p^*} \int_0^1 \|u_t^\theta(X_t) - \sigma_t \nabla \log p_t^*(X_t)\|^2 dt + \text{const} \quad (23)$$

$$= \frac{1}{2} \mathbb{E} \int_0^1 \|u_t^\theta(X_t) - \sigma_t \nabla \log p_{t|1}^{\text{base}}(X_t | X_1)\|^2 dt + \text{const}', \quad (24)$$

where in (24) $X_1 \sim p_1^$ and $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1)$, and both constants are θ -free.*

Proof. (i) The law of p^* . p^* is the Doob h -transform of p^{base} by $e^{\beta r(X_1)}$: it is Markov with drift $b_t + \sigma_t^2 \nabla \log H_t$, $H_t(x) := \mathbb{E}_{p^{\text{base}}}[e^{\beta r(X_1)} \mid X_t = x]$. By (3), $p_t^* = p_t^{\text{base}} H_t / Z$, so $\nabla \log H_t = \nabla \log p_t^* - \nabla \log p_t^{\text{base}}$ and the drift is $-f_t + \sigma_t^2 \nabla \log p_t^*$ by (1).

(ii) *Girsanov for the pair.* Both laws are diffusions with the same σ_t ; the chain rule at $t = 0$ and (6) applied to their drift gap give (23), the constant being $D_{\text{KL}}(p_0^* \parallel \mu)$.

(iii) *The conditional-mean swap.* By (4), differentiating under the integral,

$$\nabla \log p_t^*(x) = \frac{\int \nabla_x p_{t|1}^{\text{base}}(x \mid x_1) p_1^*(x_1) dx_1}{p_t^*(x)} = \mathbb{E}[\nabla_x \log p_{t|1}^{\text{base}}(x \mid X_1) \mid X_t = x], \quad (25)$$

the conditioning being under p^* , whose joint of (X_t, X_1) is exactly the sampling in (24), since $p_{\cdot|1}^* = p_{\cdot|1}^{\text{base}}$. Replacing a regression target by a random variable with the same conditional mean changes the objective by a θ -free variance. $\square$

Remark (Which target does u^θ generate?). The minimizer is $u_t^\theta = \sigma_t \nabla \log p_t^*$, which generates p_t^* when started at $p_0^* \propto \mu(x_0) \mathbb{E}_{p^{\text{base}}}[e^{\beta r(X_1)} \mid X_0 = x_0]$, whereas the trainee starts at μ . The gap is the constant $D_{\text{KL}}(p_0^* \parallel \mu)$ of (23): it does not affect training, and it vanishes when the noising kernel $p_{1|0}^{\text{base}}$ is memoryless — the memoryless schedule condition of Domingo-Enrich et al. (2025).

The empirical objective Draw $X^1, \dots, X^N \sim p^{\bar{u}}$ and self-normalize the weights of proposition 3.2; the weighted endpoints stand for p_1^* in (24):

$$w^i = \text{softmax}_i \left(\log \frac{dp^*}{dp^{\bar{u}}}(X^i) \right), \quad \mathcal{L}(\theta) = \sum_{i=1}^N w^i \mathbb{E}_{t, X_t} \left[\|u_t^\theta(X_t) - \sigma_t \nabla \log p_{t|1}^{\text{base}}(X_t \mid X_1^i)\|^2 \right], \quad (26)$$

with $t \sim \text{Unif}[0, 1]$ and $X_t \sim p_{t|1}^{\text{base}}(\cdot \mid X_1^i)$: the trajectory is thrown away, only the pair (X_1^i, w^i) is kept. For linear f_t the kernel is Gaussian and the target is the usual noise / velocity prediction — weighted denoising score matching, that is, weighted flow matching.

Algorithm 2 Cross-entropy training (Phillips et al., 2024)

- 1: Simulate N particles of $p^{\bar{u}}$ from $X_0^i \sim \mu$, accumulating $\log w^i = -\bar{u}_t(X_t^i)^\top \Delta W_t^i + \frac{1}{2} \|\bar{u}_t(X_t^i)\|^2 \Delta t$.
 - 2: At $t = 1$: $w^i = \text{softmax}_i(\log w^i + \beta r(X_1^i))$ ($\log Z$ drops in the softmax); keep the pairs (X_1^i, w^i) .
 - 3: Draw $t \sim \text{Unif}[0, 1]$, $X_t \sim p_{t|1}^{\text{base}}(\cdot \mid X_1^i)$, and take a gradient step on (26).
 - 4: Optionally set $\bar{u} \leftarrow u^\theta$ and repeat. ($\bar{u} = 0$: an offline batch of the pretrained model.)
-

3.3 Two Extensions

Proposition 3.4 (Extension I: weights from any test-time sampler). *Any (X_1^i, w^i) produced by algorithm 1 — any interpolation r_t , any control u , resampling included — is a valid weighted batch for (26).*

Proof. $\mathcal{L}$ in (26) consumes only the endpoint law: it needs (X_1, w) with $\mathbb{E}[wF(X_1)]/\mathbb{E}[w] = \mathbb{E}_{p_1^*}[F]$ for every F , which is exactly (12). The path measures differ, the endpoint marginal does not; the choices move variance, not correctness. $\square$

Theorem 3.5 (Extension II: the proximal cross entropy). *Let $p^{(k)}$ be the current model (control $u^{(k)}$, $p^{(0)} = p^{\text{base}}$, $u^{(0)} = 0$) and $\gamma_k > 0$. The Kullback–Leibler proximal step and its weight are*

$$p^{(k+1)} := \arg \min_p \left\{ D_{\text{KL}}(p \parallel p^*) + \frac{1}{\gamma_k} D_{\text{KL}}(p \parallel p^{(k)}) \right\} \implies dp^{(k+1)} \propto (dp^*)^{\alpha_k} (dp^{(k)})^{1-\alpha_k}, \quad \alpha_k = \frac{\gamma_k}{1+\gamma_k}, \quad (27)$$

$$X \sim p^{(k)} : \quad \log w = \alpha_k \left(\beta r(X_1) - \int_0^1 u_t^{(k)\top} dW_t - \frac{1}{2} \int_0^1 \|u_t^{(k)}\|^2 dt \right) + \text{const.} \quad (28)$$

Proof. Write $\alpha = \alpha_k$. Then $D_{\text{KL}}(p \| p^*) + \frac{1}{\gamma} D_{\text{KL}}(p \| p^{(k)}) = \frac{1}{\alpha} [\int p \log p - \int p(\alpha \log p^* + (1 - \alpha) \log p^{(k)})] = \frac{1}{\alpha} D_{\text{KL}}(p \| q) + \text{const}$ with $dq \propto (dp^*)^\alpha (dp^{(k)})^{1-\alpha}$: the two divergences merge into one, minimized at $p = q$. For the weight, $\frac{dp^{(k+1)}}{dp^{(k)}} \propto \left(\frac{dp^*}{dp^{(k)}}\right)^\alpha$ by (27), and the bracket is $\log \frac{dp^*}{dp^{(k)}}$ up to a constant by proposition 3.2 applied to $\bar{u} = u^{(k)}$. $\square$

Algorithm 3 Proximal cross-entropy training (Guo et al., 2026)

- 1: **outer loop** ($k = 0, 1, 2, \dots$): sample $X^1, \dots, X^N \sim p^{(k)}$ with the control $u^{(k)}$, accumulating $\log w^i$ by (28).
 - 2: Set $w^i = \text{softmax}_i(\log w^i)$; keep the pairs (X_1^i, w^i) — they are a batch for the inner target $p^{(k+1)}$.
 - 3: **inner loop**: with the data fixed, take gradient steps on (26) until converged.
 - 4: Set $u^{(k+1)} \leftarrow u^\theta$ and return to step 1.
-

Remark (Why the proximal step). One round of algorithm 2 asks the weights to bridge $p^{\bar{u}}$ all the way to p^* ; the softmax then concentrates on a few particles. The proximal step tempers the same weight by α_k and moves the model a little at a time, keeping N_{eff} healthy. The cost: many outer rounds.

3.4 Langevin Dynamics and Its Weight

What Langevin gets, and what it does not The score of the target is available — $\nabla \log p_1^* = \nabla \log p_1^{\text{base}} + \beta \nabla r$, the frozen network plus the reward gradient — so the Langevin diffusion

$$\begin{aligned} dY_s &= \nabla \log p_1^*(Y_s) ds + \sqrt{2} dB_s && (\text{invariant law } p_1^*), \\ Y^{k+1} &= Y^k + \delta \nabla \log p_1^*(Y^k) + \sqrt{2\delta} \zeta^k, && \zeta^k \sim \mathcal{N}(0, I) \quad (\text{its Euler step}). \end{aligned} \quad (29)$$

can be run on the target itself. Its *density* cannot: a diffusion model returns a score, never a normalizing constant, so $\log p_1^{\text{base}}$ — and with it every Metropolis–Hastings ratio — is out of reach. Everything below turns on that asymmetry.

Markov chain Monte Carlo alone Run (29) to stationarity from any initialization and the batch is unweighted, $w \equiv 1$: there is nothing to compute, and (26) is ordinary denoising on those samples. What is paid instead is mixing. The reward sharpens and separates the modes of p_1^{base} — that is what $e^{\beta r}$ is for — and a chain crosses between them in a time no one can bound. The failure is also silent: an unmixed chain reports nothing, whereas the importance weight of proposition 3.2 is exact at every N and its degeneracy is visible in N_{eff} . The two errors are of different kinds, and the constructions below buy one with the other.

Proposition 3.6 (A weighted batch survives an invariant kernel). *Let (X, w) satisfy (12), let M be a Markov kernel with $p_1^* M = p_1^*$, and let $X' \sim M(X, \cdot)$ be drawn conditionally on X alone. Then (X', w) satisfies (12): the weight is not touched by the move.*

Proof. Write $(MF)(x) := \int F(y) M(x, dy)$. Then

$$\mathbb{E}[w F(X')] = \mathbb{E}[w (MF)(X)] = \mathbb{E}[w] \mathbb{E}_{p_1^*}[MF] = \mathbb{E}[w] \mathbb{E}_{p_1^*}[F],$$

the middle step by the hypothesis on (X, w) and the last by invariance. $\square$

A neural sampler, then a little Langevin This is the recipe the proposition licenses: simulate $p^{\bar{u}}$, form $\log \frac{dp^*}{dp^{\bar{u}}}$ by proposition 3.2, then move every particle a few Langevin steps and keep the weight it already carries. The refined pairs go straight into (26) — proposition 3.4 never asked where the batch came from (Chen et al., 2025; Akhond-Sadeh et al., 2024).

Remark (Exactness is the whole question). proposition 3.6 needs *exact* invariance. The Metropolis-adjusted chain has it, and needs density ratios: available when the target is an energy, $p_1^* \propto e^{-E+\beta r}$ (the neural-sampler setting), unavailable when the base is a pretrained model. The unadjusted chain (29) needs only the score,

and leaves invariant not p_1^* but a law at distance $O(\delta)$ from it (Roberts and Tweedie, 1996) — a bias no weight repairs, since the law of the iterate has no closed form. Between the two: the accept ratio asks for a log-density *difference* between two nearby points, and that is a line integral of the score,

$$\log \frac{p_1^*(y)}{p_1^*(x)} = \int_0^1 \nabla \log p_1^*(x + s(y-x))^\top (y-x) ds \approx \frac{1}{2} (\nabla \log p_1^*(x) + \nabla \log p_1^*(y))^\top (y-x), \quad (30)$$

with trapezoid error $O(\|y-x\|^3) = O(\delta^3/2)$ per proposal — smaller than the bias it removes, not zero, and the extra score call is the one the reverse proposal already needs.

Proposition 3.7 (Annealed Langevin: the weight). *Anneal the reward: $\pi_k \propto p_1^{\text{base}} e^{\beta_k r}$ with $0 \leq \beta_0 < \dots < \beta_K = \beta$, so $\pi_K = p_1^*$. Let (X^0, w_0) satisfy (12) for π_0 , and for $k = 1, \dots, K$ reweight, then move with any π_k -invariant kernel M_k :*

$$\log w_k = \log w_{k-1} + (\beta_k - \beta_{k-1}) r(X^{k-1}), \quad X^k \sim M_k(X^{k-1}, \cdot). \quad (31)$$

Then (X^K, w_K) satisfies (12) for p_1^ . Only the reward enters the increments: the intractable p_1^{base} cancels in every ratio.*

Proof. On the extended space, compare the law of the chain with a backward construction of the target,

$$Q(x^{0:K}) = q(x^0) \prod_{k=1}^K M_k(x^{k-1}, x^k), \quad \Pi(x^{0:K}) = \pi_K(x^K) \prod_{k=1}^K L_{k-1}(x^k, x^{k-1}),$$

with $L_{k-1}(x^k, x^{k-1}) := \pi_k(x^{k-1}) M_k(x^{k-1}, x^k) / \pi_k(x^k)$, a Markov kernel exactly because M_k leaves π_k invariant; the x^K -marginal of Π is π_K . In the ratio each M_k cancels, and what is left telescopes:

$$\frac{\Pi}{Q}(x^{0:K}) = \frac{\pi_K(x^K)}{q(x^0)} \prod_{k=1}^K \frac{\pi_k(x^{k-1})}{\pi_k(x^k)} = \frac{\pi_0(x^0)}{q(x^0)} \prod_{k=1}^K \frac{\pi_k(x^{k-1})}{\pi_{k-1}(x^{k-1})}, \quad (32)$$

which is w_K of (31) once $\pi_k / \pi_{k-1} = e^{(\beta_k - \beta_{k-1})r}$ is inserted and $w_0 = \pi_0 / q$ at x^0 . Hence $\mathbb{E}_Q[w_K F(X^K)] \propto \mathbb{E}_{\pi_K}[F]$, and self-normalizing removes the constant (Neal, 2001; Del Moral et al., 2006). $\square$

Remark (Two familiar special cases). $\beta_0 = \beta$: every increment is 1 and proposition 3.7 is proposition 3.6. $K = 1$ with $\beta_0 = 0, \beta_1 = \beta$, started from a model batch ($q = p_1^{\text{base}}, w_0 \equiv 1$): the weight is $e^{\beta r(X^0)}$ — the offline regime of algorithm 2, with one Langevin sweep appended. In between, the schedule β_k does in the chain index what the interpolation β_t of (13) does in diffusion time, and for the same reason: move the target a little at a time, so that no single ratio carries all the variance.

Correctors inside the trajectory The same argument applies before $t = 1$. Cutting the proof of theorem 2.2 at time t rather than at 1 gives $\log w_t = r_t(X_t) + \log \frac{dp^{\text{base}}}{dp^u} \Big|_{[0,t]}$, so for every test function F

$$\mathbb{E}_{p^u}[w_t F(X_t)] = \mathbb{E}_{p^{\text{base}}}[e^{r_t(X_t)} F(X_t)] = \int p_t^{\text{base}} e^{r_t} F : \quad (33)$$

the self-normalized particles at time t represent $\propto p_t^{\text{base}} e^{r_t}$, which is what licenses the resampling step of algorithm 1. That law has an available score too, $\nabla \log p_t^{\text{base}} + \nabla r_t$, so Langevin steps at time t leave it invariant and proposition 3.6 carries the weights through unchanged: predictor–corrector sampling (Song et al., 2021) and Feynman–Kac correctors (Skreta et al., 2025) are this proposition applied at every noise level.

Remark (Resample, then move). A move by an invariant kernel does not reduce weight variance — the weights are untouched. It pays after a resampling step, which equalizes the weights but duplicates particles: the move separates the duplicates without leaving the target (Gilks and Berzuini, 2001). Resampling alone loses diversity, Langevin alone loses the guarantee; the pair keeps both.

Algorithm 4 Cross-entropy training on a Langevin-refined batch

- 1: Simulate N particles of $p^{\bar{u}}$ with their log-weights ([algorithm 2](#)), or take any weighted batch from [algorithm 1](#).
 - 2: Resample by $w^i = \text{softmax}_i(\log w^i)$ and reset $w^i = 1/N$.
 - 3: Move each particle K steps of [\(29\)](#) targeting p_1^* — Metropolis-adjusted if the density is available, unadjusted at the cost of an $O(\delta)$ bias otherwise — annealing by [\(31\)](#) if the batch is still far from the target.
 - 4: Take gradient steps on [\(26\)](#) with the refined pairs (X_1^i, w^i) ; optionally set $\bar{u} \leftarrow u^\theta$ and repeat.
-

4 Relative Entropy and Adjoint Matching

4.1 The Stochastic Control Problem

Theorem 4.1 (The reverse KL is a control problem). *For the proposal [\(5\)](#),*

$$D_{\text{KL}}(p^u \parallel p^*) = \mathbb{E}_{p^u} \left[\int_0^1 \frac{1}{2} \|u_t(X_t)\|^2 dt - \beta r(X_1) \right] + \log Z. \quad (34)$$

Its minimizer is p^ itself, attained at $u_t^* = \sigma_t \nabla \log H_t$, $H_t(x) = \mathbb{E}_{p^{\text{base}}} [e^{\beta r(X_1)} \mid X_t = x]$.*

Proof. $\log \frac{dp^u}{dp^*} = \log \frac{dp^u}{dp^{\text{base}}} - \beta r(X_1) + \log Z$; take $\mathbb{E}_{p^u}$ and use [\(6\)](#), the stochastic integral having mean zero. The right-hand side is minimized at $D_{\text{KL}} = 0$, i.e., $p^u = p^*$, whose drift is $b_t + \sigma_t^2 \nabla \log H_t$ (step (i) of [theorem 3.3](#)). $\square$

Remark (The order of information). Sections 2 and 3 use r by *value* only: they weight, they never differentiate the reward. [\(34\)](#) is minimized by a gradient method through the simulation, so ∇r enters — first order. When ∇r is available it is the richer signal per sample; the price is that the trajectories must come from the current model.

4.2 The Adjoint System

Write $g := -\beta r$, so that the objective of [\(34\)](#) is $\mathbb{E}[\int_0^1 \frac{1}{2} \|u_t\|^2 dt + g(X_1)]$ and $H_t = \mathbb{E}_{p^{\text{base}}} [e^{-g(X_1)} \mid X_t]$.

Theorem 4.2 (The adjoint representation of the optimal control). *Fix a trajectory and let $a_s := (\partial X_1 / \partial X_s)^\top \nabla g(X_1)$ be computed along the reference flow with the noise frozen; equivalently*

$$\frac{da_s}{ds} = -(\nabla_x b_s(X_s))^\top a_s, \quad a_1 = \nabla g(X_1) \quad (a \text{ backward ODE, no noise}). \quad (35)$$

Then the optimal control of [theorem 4.1](#) is the conditional mean of the adjoint state under the optimal process,

$$u_t^*(x) = -\sigma_t \mathbb{E}_{p^*} [a_t \mid X_t = x]. \quad (36)$$

Proof. (i) *The adjoint ODE.* With σ_t state-independent, the first variation of the flow of $dX_s = b_s(X_s)dt + \sigma_s dW_s$ obeys $\frac{d}{ds}(\partial X_s / \partial X_t) = \nabla_x b_s(X_s) \partial X_s / \partial X_t$. Differentiating $\partial X_1 / \partial X_t = (\partial X_1 / \partial X_s)(\partial X_s / \partial X_t)$ in s and transposing gives [\(35\)](#).

(ii) *Differentiate the potential.* Freezing the noise, $X_1 = \Phi_{t \rightarrow 1}(x)$ is a differentiable function of the initial point, so

$$\nabla_x H_t(x) = \nabla_x \mathbb{E} [e^{-g(\Phi_{t \rightarrow 1}(x))}] = \mathbb{E} [e^{-g(X_1)} (-a_t) \mid X_t = x], \quad (37)$$

the expectation being over p^{base} . Divide by $H_t(x)$: the factor $e^{-g(X_1)}/H_t(x)$ is exactly the density of p^* over p^{base} for the future conditioned on $X_t = x$, so $\nabla \log H_t(x) = -\mathbb{E}_{p^*}[a_t \mid X_t = x]$. Multiply by σ_t and use [theorem 4.1](#). $\square$

Corollary 4.3 (Adjoint matching). *Let $\bar{u} = \text{sg}(u^\theta)$ (the trajectory and the adjoint carry no gradient) and*

$$\mathcal{L}_{\text{AM}}(\theta) = \mathbb{E}_{X \sim p^{\bar{u}}} \left[\int_0^1 \|u_t^\theta(X_t) + \sigma_t a_t\|^2 dt \right], \quad \text{a from (35) along the simulated } X. \quad (38)$$

At $\bar{u} = u^$ the trajectories are p^* -distributed and u^* is the exact minimizer of (38): the optimal control is a fixed point of “simulate, then regress”. Off-policy trajectories are corrected by the weights of [proposition 3.2](#).*

Proof. A least-squares problem is minimized at the conditional mean of its target: $\arg \min_u \mathbb{E} \|u_t(X_t) + \sigma_t a_t\|^2 = -\sigma_t \mathbb{E}[a_t \mid X_t]$, which is u^* by (36) when the sampling law is p^* . Weighting the trajectories by $dp^*/dp^{\bar{u}}$ restores that law for any $\bar{u}$. $\square$

Remark (The general form). For a general control problem $\min_u \mathbb{E}[\int_0^1 L(t, X_t, u_t) dt + g(X_1)]$ subject to $dX_t = b(t, X_t, u_t) dt + \sigma_t dW_t$, with Hamiltonian $H(t, x, u, y) = L(t, x, u) + b(t, x, u)^\top y$, the same argument gives

$$\mathcal{L}_{\text{AM}}(\theta) = \frac{1}{2} \mathbb{E}_{X \sim p^{\bar{u}}} \left[\int_0^1 \|\nabla_u H(t, X_t, u_t^\theta, \tilde{Y}_t)\|^2 dt \right], \quad d\tilde{Y}_t = -\nabla_x H(t, X_t, \bar{u}_t, \tilde{Y}_t) dt, \quad \tilde{Y}_1 = \nabla g(X_1); \quad (39)$$

(38) is the case $L = \frac{1}{2}\|u\|^2$, $b = b_t + \sigma_t u$, where $\nabla_u H = u + \sigma_t y$. Dropping the control’s own Jacobian from $\nabla_x H$ — the *lean* adjoint (35) — is what [theorem 4.2](#) justifies, and it is exact under the memoryless condition discussed after [theorem 3.3](#) ([Domingo-Enrich et al., 2025](#)). [proposition 5.8](#) is a worked instance with a state-dependent running cost.

5 Stochastic Density Control

5.1 The Problem

Definition 5.1 (The density-control target). Two pretrained models with marginals ν_t^A, ν_t^B share the noising pair (f_t, σ_t) and the prior $\nu_0^A = \nu_0^B = p_0$; write their scores $v_t^A := \sigma_t \nabla \log \nu_t^A$ and $v_t^B := \sigma_t \nabla \log \nu_t^B$. For $\gamma > 0$, sample

$$\rho_{\text{target}} \propto \frac{(\nu_1^B)^{\gamma+1}}{(\nu_1^A)^\gamma}. \quad (40)$$

Instances One template covers several practices: classifier-free guidance ([Ho and Salimans, 2022](#)) (B conditional, A unconditional, γ the guidance weight — (40) is the density that guided sampling is *aiming* at); machine unlearning and model negotiation (push away from A , toward B); and, when B is given by an energy rather than by a network, binder design and mode discovery.

It is a reward tilt Take ν^B as the base model: then (40) is [definition 1.1](#) with

$$r = \log \frac{\nu_1^B}{\nu_1^A}, \quad \beta = \gamma. \quad (41)$$

Example 5.2 (The reward that cannot be evaluated). A diffusion model gives its score, never its density: r of (41) is not evaluable pointwise, while $\nabla r = \nabla \log \nu_1^B - \nabla \log \nu_1^A$ is immediate from the two scores — the exact reverse of [example 2.4](#), and a case the order-of-information discussion after [theorem 4.1](#) does not cover. Sections 2 and 3 need the *value* of r , so they look inapplicable. They are not: the value is unavailable at a point, but computable *along a simulated path* ([lemma 5.3](#)).

What is given Two regimes, treated separately below.

- **Two scores.** Both v^A and v^B are networks; both models can be simulated.
- **One score and one energy.** v^A is a network, while B is known only as an unnormalized terminal density $\nu_1^B \propto e^{-E/\tau}$, with E and ∇E evaluable. Here $\log \nu_1^B(X_1)$ is available at the endpoint, and nothing about B is simulable.

The path-space lift Let p^r be the reference $dX_t = -f_t(X_t)dt + \sigma_t dW_t$, $X_0 \sim p_0$, and p^u its σ -scaled control (5). The target path measure and its unnormalized weight are

$$dp^*(X) = \frac{1}{Z} \frac{(\nu_1^B)^{\gamma+1}}{(\nu_1^A)^\gamma p_1^r} (X_1) dp^r(X), \quad \tilde{\omega}(X) := \frac{(\nu_1^B)^{\gamma+1}}{(\nu_1^A)^\gamma p_1^r} (X_1) \frac{dp^r}{dp^u}(X) = Z \frac{dp^*}{dp^u}(X). \quad (42)$$

The tilt depends on X_1 only, so the endpoint marginal of p^* is exactly (40), by the disintegration of (4).

5.2 The Weight Along the Path

Lemma 5.3 (The log-density along the path). *Along $dX_t = (-f_t + \sigma_t u_t)dt + \sigma_t dW_t$, for $\nu = \nu^A$ or ν^B with score $v = \sigma \nabla \log \nu$,*

$$\log \nu_1(X_1) = \log \nu_0(X_0) + \int_0^1 \left[\nabla \cdot f_t + v_t^\top u_t - \frac{1}{2} \|v_t\|^2 \right] (X_t) dt + \int_0^1 v_t(X_t)^\top dW_t. \quad (43)$$

Proof. Itô on $\log \nu_t(X_t)$ gives $[\partial_t \log \nu_t + (-f_t + \sigma_t u_t) \cdot \nabla \log \nu_t + \frac{\sigma_t^2}{2} \Delta \log \nu_t] dt + v_t^\top dW_t$. Substitute $\partial_t \log \nu_t = \partial_t \nu_t / \nu_t$ from the model's Fokker–Planck equation $\partial_t \nu_t = -\nabla \cdot ((-f_t + \sigma_t v_t) \nu_t) + \frac{\sigma_t^2}{2} \Delta \nu_t$, expand with $\Delta \nu / \nu = \Delta \log \nu + \|\nabla \log \nu\|^2$: the term $\sigma_t \nabla \cdot v_t = \sigma_t^2 \Delta \log \nu_t$ cancels the two Laplacians, and what is left is $\nabla \cdot f_t + v_t^\top u_t - \|v_t\|^2 + \frac{1}{2} \|v_t\|^2$. $\square$

Proposition 5.4 (The weight, two scores). *Along p^u ,*

$$\begin{aligned} \log \tilde{\omega}(X) = \log \frac{p_0(X_0)}{p_1^r(X_1)} + \int_0^1 \left[\nabla \cdot f_t - \frac{1}{2} \|u_t - [v_t^B + \gamma(v_t^B - v_t^A)]\|^2 + \frac{\gamma(\gamma+1)}{2} \|v_t^B - v_t^A\|^2 \right] dt \\ + \int_0^1 \left([v_t^B + \gamma(v_t^B - v_t^A)] - u_t \right)^\top dW_t. \end{aligned} \quad (44)$$

Proof. $\log \tilde{\omega} = (\gamma+1) \log \nu_1^B(X_1) - \gamma \log \nu_1^A(X_1) - \log p_1^r(X_1) + \log \frac{dp^r}{dp^u}(X)$. Apply lemma 5.3 to each model and combine: the starts give $(\gamma+1) \log p_0 - \gamma \log p_0 = \log p_0(X_0)$, the divergences give $\nabla \cdot f_t$, the dW -terms give $v_t^B + \gamma(v_t^B - v_t^A)$ and the u -terms its inner product with u_t , while $-\frac{1}{2} [(\gamma+1) \|v^B\|^2 - \gamma \|v^A\|^2] = -\frac{1}{2} \|v^B + \gamma(v^B - v^A)\|^2 + \frac{\gamma(\gamma+1)}{2} \|v^B - v^A\|^2$. Finally $\log \frac{dp^r}{dp^u} = -\int u^\top dW - \int \frac{1}{2} \|u\|^2 dt$ (theorem 1.2) completes the square. $\square$

Corollary 5.5 (Steering by the guidance drift). *At $u_t = v_t^B + \gamma(v_t^B - v_t^A)$ — classifier-free guidance — the quadratic mismatch and the stochastic integral both vanish:*

$$\log \tilde{\omega}(X) = \log \frac{p_0(X_0)}{p_1^r(X_1)} + \int_0^1 \left[\nabla \cdot f_t + \frac{\gamma(\gamma+1)}{2} \|v_t^B - v_t^A\|^2 \right] dt. \quad (45)$$

Proof. Put $u_t = v_t^B + \gamma(v_t^B - v_t^A)$ in (44). $\square$

Remark (Guidance is the steering). The drift $v_t^B + \gamma(v_t^B - v_t^A)$ is classifier-free guidance, and corollary 5.5 says what it does: it kills the stochastic integral of the weight — the role U_t plays in theorem 2.2. It is not literally that U_t , which would carry $-\sigma_t \nabla \log p_t^r$ as well: the factor $1/p_1^r(X_1)$ stays outside the path expansion, because p_t^r is a reference marginal and lemma 5.3 needs a model one. What guided sampling omits is the

rest of (45): the drift $\frac{\gamma(\gamma+1)}{2}\|v_t^B - v_t^A\|^2$, small exactly where the two models agree, and the endpoint ratio $p_0(X_0)/p_1^r(X_1)$.

Proposition 5.6 (The weight, one score and one energy). *Let $\nu_1^B \propto e^{-E/\tau}$. Then, along p^u ,*

$$\begin{aligned} \log \tilde{w}(X) = & (\gamma + 1) \log \nu_1^B(X_1) - \log p_1^r(X_1) - \gamma \log p_0(X_0) \\ & - \int_0^1 \left[\gamma \nabla \cdot f_t + \frac{1}{2} \|u_t + \gamma v_t^A\|^2 - \frac{\gamma(\gamma+1)}{2} \|v_t^A\|^2 \right] dt - \int_0^1 (u_t + \gamma v_t^A)^\top dW_t, \end{aligned} \quad (46)$$

with $(\gamma + 1) \log \nu_1^B(X_1) = -\frac{\gamma+1}{\tau} E(X_1) + \text{const.}$ At $u = -\gamma v^A$ the mismatch and the stochastic integral vanish.

Proof. Same split, but only ν^A needs lemma 5.3: the endpoint value $\log \nu_1^B(X_1)$ is given. Collect the dt -terms, $-\gamma \nabla \cdot f_t - \gamma (v_t^A)^\top u_t + \frac{\gamma}{2} \|v_t^A\|^2 - \frac{1}{2} \|u_t\|^2$, and complete the square with $-\frac{1}{2} \|u + \gamma v^A\|^2 = -\frac{1}{2} \|u\|^2 - \gamma u^\top v^A - \frac{\gamma^2}{2} \|v^A\|^2$; the dW -terms are $-\gamma v_t^A - u_t$. $\square$

5.3 The Algorithms

Algorithm 5 Density control at test time: the weighted sampler (two scores)

- 1: Draw $X_0^i \sim p_0$, set $\log w^i = \log p_0(X_0^i)$, and choose a control (the guidance drift below; any other u uses (44)).
 - 2: Euler step the state with the guidance drift, $X_{t+\Delta t}^i = X_t^i + (-f_t + \sigma_t [v_t^B + \gamma(v_t^B - v_t^A)])(X_t^i) \Delta t + \sigma_t \Delta W_t^i$: two network calls.
 - 3: Accumulate, with no stochastic integral (corollary 5.5): $\log w^i += [\nabla \cdot f_t + \frac{\gamma(\gamma+1)}{2} \|v_t^B - v_t^A\|^2](X_t^i) \Delta t$.
 - 4: Optionally resample the particles by their weights and reset them.
 - 5: At $t = 1$: $w^i = \text{softmax}_i(\log w^i - \log p_1^r(X_1^i))$; report the weighted particles (X_1^i, w^i) .
-

Algorithm 6 Density control by training: one model for ρ_{target}

- 1: Run algorithm 5 (or any control, weighted by (44) / (46)) and keep the pairs (X_1^i, w^i) .
 - 2: Draw $t \sim \text{Unif}[0, 1]$, $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1^i)$ — the noising kernel shared by both models — and take a gradient step on the weighted denoising objective (26).
 - 3: Optionally set $\bar{u} \leftarrow u^\theta$ and repeat; theorem 3.5 tempers the weights if the softmax concentrates.
-

Remark (Why the training step is legitimate). algorithm 5 is an instance of algorithm 1, so proposition 3.4 applies verbatim: its output is a weighted batch for ρ_{target} , and (26) consumes nothing else. The trained control samples (40) in one pass — no two networks at inference, no guidance scale to tune.

Remark (Splitting the weight: a recursive Feynman–Kac flow). For *any* strictly positive ξ_1 — in practice a third score model, for instance the current trainee —

$$\tilde{w}(X) = \underbrace{\left(\frac{\xi_1}{\nu_1^A} \right)^\gamma}_{\text{carried and resampled}} \underbrace{\left(\frac{\nu_1^B}{\xi_1} \right)^{\gamma+1} \frac{\xi_1}{p_1^r}}_{\text{terminal}} \frac{dp^r}{dp^u}(X) \quad (X_1\text{-arguments suppressed}), \quad (47)$$

since the powers of ξ_1 cancel. Each factor is computable along the path by lemma 5.3. Splitting matters because resampling acts on the first factor early — while the particles are still exchangeable — and only the second is left to the endpoint softmax; the closer ξ is to the target, the smaller both.

5.4 The Control Formulation

Proposition 5.7 (The control problems of density control). *Up to constants in u , $D_{\text{KL}}(p^u \| p^\star) = \mathbb{E}_{p^u}[-\log \tilde{\omega}] + \log Z$ reads*

$$(two\ scores) \quad \min_u \mathbb{E}_{p^u} \left[\log \frac{p_1^r(X_1)}{p_0(X_0)} + \int_0^1 \left(\frac{1}{2} \|u_t - [v_t^B + \gamma(v_t^B - v_t^A)]\|^2 - \frac{\gamma(\gamma+1)}{2} \|v_t^B - v_t^A\|^2 \right) dt \right], \quad (48)$$

$$(energy) \quad \min_u \mathbb{E}_{p^u} \left[\log p_1^r(X_1) + \frac{\gamma+1}{\tau} E(X_1) + \int_0^1 \left(\frac{1}{2} \|u_t + \gamma v_t^A\|^2 - \frac{\gamma(\gamma+1)}{2} \|v_t^A\|^2 \right) dt \right], \quad (49)$$

for affine f_t ; in general $-\int_0^1 \nabla \cdot f_t dt$ (respectively $\gamma \int_0^1 \nabla \cdot f_t dt$) joins the integrand.

Proof. Take $\mathbb{E}_{p^u}$ of $-\log \tilde{\omega}$ in (44) and (46): the stochastic integrals are p^u -martingales, hence mean zero. For affine f_t the divergence is x -free, and $\mathbb{E}_{p^u}[\gamma \log p_0(X_0)]$ is a constant because every p^u starts at p_0 ; $\log p_1^r(X_1)$ is not — for affine f_t and Gaussian p_0 it is an explicit Gaussian log-density. $\square$

Remark (Both objectives are evaluable). Neither problem mentions ν_1^A or ν_1^B : the running costs use only the scores, and the terminal costs only E and the reference’s own density. This is [theorem 4.1](#) for a reward whose value is unavailable — the zero-order/first-order trade discussed after [theorem 4.1](#), made in the only direction available here.

Proposition 5.8 (The adjoint system of density control). *Write $\alpha_t := u_t + \gamma v_t^A$, so that the state equation of (49) is $dX_t = [-f_t + \sigma_t(\alpha_t - \gamma v_t^A)]dt + \sigma_t dW_t$. This is the general control problem stated after [corollary 4.3](#) with*

$$L(t, x, \alpha) = \frac{1}{2} \|\alpha\|^2 - \frac{\gamma(\gamma+1)}{2} \|v_t^A(x)\|^2, \quad g(x) = \frac{\gamma+1}{\tau} E(x) + \log p_1^r(x),$$

whose Hamiltonian gradients are $\nabla_\alpha H = \alpha + \sigma_t y$ and $\nabla_x H = -\nabla_x [f_t^\top y + \gamma \sigma_t (v_t^A)^\top y + \frac{\gamma(\gamma+1)}{2} \|v_t^A\|^2]$. The lean adjoint matching loss is therefore

$$\begin{aligned} \mathcal{L}_{\text{LAM}}(\theta) &= \frac{1}{2} \mathbb{E}_{X \sim p^{\text{sg}}(\alpha^\theta)} \left[\int_0^1 \|\alpha_t^\theta(X_t) + \sigma_t \tilde{Y}_t\|^2 dt \right], \\ d\tilde{Y}_t &= \nabla_x \left[f_t(X_t)^\top \tilde{Y}_t + \gamma \sigma_t v_t^A(X_t)^\top \tilde{Y}_t + \frac{\gamma(\gamma+1)}{2} \|v_t^A(X_t)\|^2 \right] dt, \quad \tilde{Y}_1 = \nabla g(X_1). \end{aligned} \quad (50)$$

Proof. Substitute L , $b(t, x, \alpha) = -f_t + \sigma_t(\alpha - \gamma v_t^A)$ and g into $H = L + b^\top y$ of the general form after [corollary 4.3](#) and differentiate; $\nabla_x H$ is taken at frozen α — the lean adjoint. The three terms of $\nabla_x[\cdot]$ are vector–Jacobian products through f_t and the frozen score network. $\square$

Algorithm 7 Density control by adjoint matching (energy regime) ([Domingo-Enrich et al., 2025](#); [Liu et al., 2025](#))

- 1: Simulate X from the current control with the gradient detached, $dX_t = [-f_t + \sigma_t(\text{sg}(\alpha_t^\theta) - \gamma v_t^A)]dt + \sigma_t dW_t$, $X_0 \sim p_0$, and cache the trajectory.
 - 2: Set $\tilde{Y}_1 = \frac{\gamma+1}{\tau} \nabla E(X_1) + \nabla \log p_1^r(X_1)$ and integrate the adjoint of (50) backward along the cached path: one vector–Jacobian product per step.
 - 3: Take a gradient step on (50); the sampler’s control is $u^\theta = \alpha^\theta - \gamma v^A$.
 - 4: Repeat. Off-policy trajectories are corrected by the weights (46), as in [corollary 4.3](#).
-

Remark (The other regime). With two scores the same reparameterization works with $\alpha_t := u_t - [v_t^B + \gamma(v_t^B - v_t^A)]$ and $g = \log p_1^r$: the drift becomes $-f_t + \sigma_t(\alpha_t + [v_t^B + \gamma(v_t^B - v_t^A)])$, and the guidance drift replaces $-\gamma v_t^A$ everywhere in [proposition 5.8](#). Which route to prefer is the zero-order/first-order trade once

more: [algorithm 5](#) is zero-order in the two models and needs no backward pass; [algorithm 7](#) differentiates them and pays for it with on-policy simulation.

6 The Bridge Matching Sampler

Notations This section stands on its own and rebinds several letters. μ is still a Gaussian prior — drawable, with $\nabla \log \mu$ in closed form — but it is now one end of an *interpolant* rather than the noise end of a pretrained model, and there is no pretrained model at all; $\nu \propto e^{-E}$ is the target, of which only the energy E and the force $-\nabla E$ may be evaluated. Points are lower case: x_0, x_1 and x_t are the two endpoints and the interpolant point, not processes. Rebound: γ_t is the interpolant schedule (not the proximal parameter γ_k of [theorem 3.5](#), nor the exponent γ of Section 5); b_t and s_t are the interpolant’s velocity and score (not the pretrained drift of (1)); g_t is a free noise level (not the terminal cost $g = -\beta r$ of Section 4); c_t blends the score’s two labels. New: b^θ and s^θ are the two learned fields — the only shorthands in the section — p_t is the law of x_t , and η is a damping weight.

6.1 The Sampling Problem

Definition 6.1 (Sampling from an energy). Given $E : \mathbb{R}^d \rightarrow \mathbb{R}$ and its gradient — and no sample of the target, no data, no pretrained model —

$$\nu(x) \propto e^{-E(x)}, \quad \text{produce } x \sim \nu. \quad (51)$$

Remark (What is different here). Sections 2–4 all begin with a pretrained p^{base} and tilt it: the reward is a correction to something that already samples, and every method carries dp^*/dp^u in some form — as a weight (§2), as an importance weight for training (§3), or minimized in u (§4). Here the target is given outright by its energy and there is nothing to tilt. What survives is the mechanism of [corollary 4.3](#) — simulate, read a label off the energy, regress — with the weights and the control cost gone.

What is simulated The sampler is one stochastic differential equation carrying two learned fields,

$$dx_t = [b_t^\theta(x_t) + g_t^2 s_t^\theta(x_t)] dt + \sqrt{2} g_t dW_t, \quad x_0 \sim \mu, \quad (52)$$

with b^θ the learned velocity, s^θ the learned score, and $g_t \geq 0$ a noise level we choose. [Section 6.2](#) says what the two fields must be for $x_1 \sim \nu$; [section 6.3](#) turns each of them into a least squares; [section 6.4](#) removes the one input we do not have.

6.2 The Interpolant and its Two Fields

Definition 6.2 (The interpolant). Fix an increasing schedule γ_t with $\gamma_0 = 0$ and $\gamma_1 = 1$, and set

$$x_t := (1 - \gamma_t) x_0 + \gamma_t x_1, \quad (x_0, x_1) \sim \mu \otimes \nu \quad (\text{the two ends drawn independently}). \quad (53)$$

Write p_t for the law of x_t , so that $p_0 = \mu$ and $p_1 = \nu$ by construction ([Albergo et al., 2025](#)).

Theorem 6.3 (The marginals of the interpolant). *The law p_t of (53) solves the transport equation*

$$\partial_t p_t + \nabla \cdot (b_t p_t) = 0, \quad p_0 = \mu, \quad p_1 = \nu, \quad \text{with} \quad b_t(x) := \mathbb{E}_{\mu \otimes \nu} [\dot{\gamma}_t (x_1 - x_0) \mid x_t = x]. \quad (54)$$

Proof. Let F be a test function. Differentiating (53) under the expectation and then conditioning on x_t ,

$$\frac{d}{dt} \mathbb{E}_{\mu \otimes \nu} [F(x_t)] = \mathbb{E}_{\mu \otimes \nu} [\dot{\gamma}_t (x_1 - x_0)^\top \nabla F(x_t)] = \mathbb{E}_{\mu \otimes \nu} [b_t(x_t)^\top \nabla F(x_t)] = \int (\nabla F)^\top b_t p_t,$$

the middle step being the tower property. The left-hand side is $\int F \partial_t p_t$; integrating the right-hand side by parts gives $-\int F \nabla \cdot (b_t p_t)$, and F was arbitrary. The velocity at a point is thus the average velocity of the pairs passing through it. $\square$

Theorem 6.4 (The noise level is free). *Let $s_t := \nabla \log p_t$. For every $g_t \geq 0$, the marginals p_t of (53) solve the Fokker–Planck equation of*

$$dx_t = [b_t + g_t^2 s_t](x_t) dt + \sqrt{2} g_t dW_t, \quad x_0 \sim \mu, \quad (55)$$

and are carried by a process solving it; in particular $x_1 \sim \nu$.

Proof. For $t < 1$ the interpolant is a nondegenerate Gaussian smoothing of $\gamma_t x_1$, since $x_0 \sim \mu$ is Gaussian and independent of x_1 : p_t is a smooth positive density and s_t is finite. Now $\nabla \cdot (p_t \nabla \log p_t) = \nabla \cdot \nabla p_t = \Delta p_t$, so adding and subtracting $g_t^2 \nabla \cdot (s_t p_t)$ in (54),

$$\partial_t p_t = -\nabla \cdot (b_t p_t) = -\nabla \cdot ([b_t + g_t^2 s_t] p_t) + g_t^2 \Delta p_t,$$

which is the Fokker–Planck equation of (55): the diffusion coefficient $\sqrt{2} g_t$ contributes $\frac{1}{2}(\sqrt{2} g_t)^2 \Delta = g_t^2 \Delta$, exactly cancelling the score drift. That a solution of a Fokker–Planck equation is carried by a process solving the corresponding stochastic differential equation is the superposition principle (Ambrosio et al., 2008; Trevisan, 2016); the converse reading — an Itô process is mimicked by a Markov one with the same marginals — is Gyöngy (1986) and Brunick and Shreve (2013). $\square$

Remark (Marginals, not paths). (55) matches p_t time by time; its trajectories are *not* the straight segments of (53), and different g_t give different path measures with the same marginals. $g_t = 0$ is the probability-flow ordinary differential equation and any $g_t > 0$ adds noise the marginals do not feel. Only $t = 1$ is asked of the sampler, and every choice delivers it — so the prior may be any Gaussian, and the noise schedule is a free knob rather than a modelling commitment.

6.3 The Two Labels

The velocity is a least squares b_t is a conditional expectation by (54), so for each t

$$b_t = \arg \min_b \mathbb{E}_{\mu \otimes \nu} [\|b(x_t) - \dot{\gamma}_t(x_1 - x_0)\|^2], \quad (56)$$

the expectation over $(x_0, x_1) \sim \mu \otimes \nu$ with x_t a deterministic function of the pair. One draw of the pair supplies both the label and the point at which it is read. The score is not, on its face, a conditional expectation of anything we can evaluate; the next theorem makes it one.

Theorem 6.5 (The score of the marginal is the force at the endpoint). *For $t \in (0, 1)$ and every x ,*

$$\nabla \log p_t(x) = \mathbb{E}_{\mu \otimes \nu} \left[\frac{1}{\gamma_t} \nabla \log \nu(x_1) \mid x_t = x \right] = -\mathbb{E}_{\mu \otimes \nu} \left[\frac{1}{\gamma_t} \nabla E(x_1) \mid x_t = x \right], \quad (57)$$

and, at the other end,

$$\nabla \log p_t(x) = \mathbb{E}_{\mu \otimes \nu} \left[\frac{1}{1-\gamma_t} \nabla \log \mu(x_0) \mid x_t = x \right]. \quad (58)$$

Proof. Let ψ_t be the density of $(1 - \gamma_t)x_0$. Since x_0 and x_1 are independent,

$$p_{t|1}(x \mid x_1) = \psi_t(x - \gamma_t x_1), \quad p_t(x) = \int p_{t|1}(x \mid x_1) \nu(x_1) dx_1,$$

so the kernel depends on (x, x_1) through $x - \gamma_t x_1$ alone and $\nabla_x p_{t|1} = -\frac{1}{\gamma_t} \nabla_{x_1} p_{t|1}$. Differentiating under the integral and integrating by parts in x_1 ,

$$\nabla_x p_t(x) = -\frac{1}{\gamma_t} \int \nabla_{x_1} p_{t|1}(x \mid x_1) \nu(x_1) dx_1 = \frac{1}{\gamma_t} \int p_{t|1}(x \mid x_1) \nabla \nu(x_1) dx_1.$$

Divide by $p_t(x)$ and write $\nabla \nu = \nu \nabla \log \nu$: the factor $p_{t|1}(x \mid x_1) \nu(x_1) / p_t(x)$ is the conditional density of x_1 given $x_t = x$, which is the first equality of (57); the second is $\nabla \log \nu = -\nabla E$, in which the normalizing constant of ν disappears. (58) is the same computation with the two ends exchanged, the kernel $p_{t|0}(x \mid x_0)$ now depending on $x - (1 - \gamma_t)x_0$. $\square$

Corollary 6.6 (The blended label). *For any $c_t \in (0, 1)$,*

$$s_t = \arg \min_s \mathbb{E}_{\mu \otimes \nu} \left[\left\| s(x_t) - \left(\frac{c_t}{\gamma_t} \nabla \log \nu(x_1) + \frac{1-c_t}{1-\gamma_t} \nabla \log \mu(x_0) \right) \right\|^2 \right]. \quad (59)$$

The choice $c_t = \gamma_t^2 / (\gamma_t^2 + (1 - \gamma_t)^2)$ keeps both coefficients bounded on $[0, 1]$:

$$\frac{c_t}{\gamma_t} = \frac{\gamma_t}{\gamma_t^2 + (1 - \gamma_t)^2}, \quad \frac{1 - c_t}{1 - \gamma_t} = \frac{1 - \gamma_t}{\gamma_t^2 + (1 - \gamma_t)^2}. \quad (60)$$

Proof. Both expressions of [theorem 6.5](#) have the same conditional mean $s_t(x)$ given $x_t = x$, hence so does every convex combination of them. A least squares is minimized at the conditional mean of its target, and replacing a target by another with the same conditional mean changes the objective by a variance that does not depend on the fitted field. $\square$

Remark (Why blend at all). The endpoint label carries $1/\gamma_t$, which blows up as $t \rightarrow 0$ — exactly where the prior label is exact, since $x_t \approx x_0$ there — and the prior label carries the mirror singularity at $t \rightarrow 1$. The weight c_t moves only the variance of the label, never its mean, so (60) is a variance-reduction choice and nothing more. Note also which quantities appear: $\nabla \log \mu$ in closed form, and $\nabla \log \nu = -\nabla E$, the one thing [definition 6.1](#) grants ([De Bortoli et al., 2024](#)).

The training objective Combining (56) with (59) at the balanced c_t of (60),

$$\begin{aligned} \mathcal{L}(\theta) &= \mathbb{E}_{t, \mu \otimes \nu} \left[\left\| b_t^\theta(x_t) - \dot{\gamma}_t (x_1 - x_0) \right\|^2 \right] \\ &+ \mathbb{E}_{t, \mu \otimes \nu} \left[\left\| s_t^\theta(x_t) - \frac{\gamma_t \nabla \log \nu(x_1) + (1 - \gamma_t) \nabla \log \mu(x_0)}{\gamma_t^2 + (1 - \gamma_t)^2} \right\|^2 \right], \end{aligned} \quad (61)$$

where $t \sim \text{Unif}[0, 1]$, $(x_0, x_1) \sim \mu \otimes \nu$, and $x_t = (1 - \gamma_t)x_0 + \gamma_t x_1$ is a deterministic function of the draws, not a draw of its own. Its minimizers are the velocity and the score of [definition 6.2](#), so (55) run with them terminates at ν by [theorem 6.4](#). The cost of one training example is one force evaluation $\nabla E(x_1)$, and no simulation appears inside the loss.

6.4 Training Without a Sample of the Target

The missing endpoint (61) needs $x_1 \sim \nu$ — the one thing [definition 6.1](#) does not give. The sampler substitutes the model’s own output: x_1 is obtained by integrating (52) with the current fields. The velocity label then describes the interpolant between μ and the model’s terminal law, whatever that law is; the score label still carries the true force $-\nabla E$, now read at that endpoint. The two agree only when the terminal law is already ν .

Theorem 6.7 (The target is a fixed point). *Suppose the current fields generate a terminal law equal to ν . Then the training pairs — a fresh $x_0 \sim \mu$ and a model endpoint x_1 — are distributed as $\mu \otimes \nu$; the minimizers of (61) are the velocity b_t and the score s_t of [definition 6.2](#); and (55) run with them again has terminal law ν .*

Proof. x_0 is drawn independently of the simulation, so the pair is a product measure, and by hypothesis its second marginal is ν : this is exactly the sampling of (53). [Theorem 6.5](#) and [corollary 6.6](#) then identify the two labels of (61) as unbiased for b_t and s_t , of which they are the least-squares minimizers, and [theorem 6.4](#) closes the loop. $\square$

Remark (What the fixed point does not say). That ν reproduces itself is a consistency statement, not a convergence statement: whether it is the only fixed point, and whether the iteration reaches it from a cold start, is open. What can be said is why the iteration moves at all. The substituted endpoint is wrong, but the label read at it is not — however far x_1 sits from equilibrium, $-\nabla E(x_1)$ is the true direction of increasing target density there — so the model is corrected exactly where it puts its mass. The energy is the only source

of information about where the mass belongs, and it enters through the label, never through the samples. This is the mechanism of [corollary 4.3](#) with the adjoint state replaced by the force itself.

Remark (Bridge matching with a self-generated endpoint). Conditioned on a pair (x_0, x_1) , (53) is a bridge between the two ends, and fitting a Markov field to labels read along such bridges is bridge matching (Lecture 2). Here the coupling is the independent one $\mu \otimes \nu$ and its second marginal is supplied by the model rather than by data — hence the name of the method, and hence [theorem 6.7](#). Iterative Markovian fitting would keep resampling the coupling toward the Schrödinger one; this loop never does.

Remark (Against the adjoint Schrödinger bridge sampler). [Liu et al. \(2025\)](#) attack the same problem — ν known only by its energy — through the Schrödinger coupling, whose endpoint label is $-(\nabla E + \nabla \log \hat{\varphi}_1)(x_1)$: a second network for the potential $\hat{\varphi}_1$, and two alternating objectives. Fixing the coupling to $\mu \otimes \nu$ removes both potentials, and (59) is the label that replaces them. Given up: the optimality of the paths. Kept: $x_0 \sim \mu$ and $x_1 \sim \nu$, which is all [definition 6.1](#) asked for — together with the freedom in the prior and in the noise level that came with it.

Algorithm 8 The bridge matching sampler ([Blessing et al., 2026b](#))

- 1: **Simulate.** Draw $x_0 \sim \mu$ and integrate (52) with the current fields to $t = 1$; keep only the endpoint x_1 .
 - 2: **Label.** Evaluate the force once and store the pair $(x_1, \nabla E(x_1))$ in a replay buffer.
 - 3: **Interpolate.** Draw a fresh $x_0 \sim \mu$ and a time $t \sim \text{Unif}[0, 1]$, and set $x_t = (1 - \gamma_t)x_0 + \gamma_t x_1$.
 - 4: **Regress.** Take a gradient step on $\mathcal{L}(\theta)$ of (61), damped by (62), and return to step 1.
-

What one gradient step costs Nothing in (61) is a functional of a trajectory: x_t is a single closed-form draw and the label is one force evaluation, so no gradient ever crosses the simulation and the memory does not grow with the number of integration steps. This is the difference from [corollary 4.3](#) and [algorithm 7](#), where an adjoint state has to be integrated backward along a cached path, and it is what lets the same recipe run in thousands of dimensions.

Remark (Damping and the replay buffer). The model’s own output is its training data, so a round that moves too far poisons the next one. [Blessing et al. \(2026b\)](#) add a proximal penalty toward the previous round’s parameters θ_i ,

$$\mathcal{L}(\theta) + \eta \mathbb{E}_{t, x_0, x_1} \left[\|b_t^\theta(x_t) - b_t^{\theta_i}(x_t)\|^2 + \|s_t^\theta(x_t) - s_t^{\theta_i}(x_t)\|^2 \right], \quad (62)$$

the draws being those of [algorithm 8](#): the fields then move only a fraction of the way per round, and the buffer replays stored endpoints so that one force evaluation trains many steps. This is [theorem 3.5](#) in another guise — a proximal step on the model, there carried by the weight and here by the loss. On Gaussian mixtures with twenty modes it is what separates a sampler that keeps its modes, up to dimension 2500, from one that collapses or fails to train outright ([Blessing et al., 2026b](#)).

6.5 JANUS: the Sampler on a Lattice

The state and the network [Blessing et al. \(2026a\)](#) run [algorithm 8](#) on disordered materials. The continuous state is the displacement of every atom from its lattice site together with the log-volume of the cell; b^θ and s^θ are two heads of one equivariant graph network conditioned on temperature and chemical potential, so a single model covers a window of thermodynamic states. The label is one call to an interatomic potential per stored endpoint — energy, forces, virial. Neither the loss nor the argument changes: only the state space and the network do.

The discrete channel Which species sits at which site is a discrete variable, and it gets a masked reveal in place of an interpolant: the network’s conditional $\mathbb{P}(\text{site } i \text{ ends as } a \mid \text{what is revealed})$ is fitted against the heat-bath label $e^{-E_i(a)/k_B T} / \sum_{a'} e^{-E_i(a')/k_B T}$, where $E_i(a)$ is the energy of the generated configuration with a placed at site i . Same logic and same fixed point: self-generated states, labels from the energy. Lecture 5 develops the chain version of the continuous channel as well, with the interpolant replaced by a reference chain and the regression target by a rate.

What it buys The comparison is in energy evaluations, not in wall-clock: on Cu–Ni, Cu–Ag and Ni–Cr alloys the sampler reaches a given estimate with about three orders of magnitude fewer calls to the potential than Monte Carlo (Blessing et al., 2026a). The reason is amortization — Monte Carlo pays afresh at every thermodynamic state, while the conditioned network pays once for the whole window — and, on top of it, the weights of a single rollout give free energies, so the loop also measures itself.

Summary

	test-time scaling (§2)	cross entropy (§3)	relative entropy (§4)
what is optimized	nothing — weights only	$D_{\text{KL}}(p^* \ p^{u^\theta})$ (forward)	$D_{\text{KL}}(p^{u^\theta} \ p^*)$ (reverse)
what is simulated	any proposal p^u	any p^u , even offline	the current model
use of the reward	value r	value r	gradient ∇r
the estimate	self-normalized particles	weighted denoising	adjoint matching
what can go wrong	weight variance	weight variance (N_{eff})	on-policy simulation cost

Table 2 Three ways to sample $p_1^* \propto p_1^{\text{base}} e^{\beta r}$ from a frozen model. All three carry the same object: the Radon–Nikodym derivative of the target over what is simulated, [theorem 2.2](#). Section 5 runs all three columns on one problem, where the reward is a log-density ratio that is never evaluated pointwise.

One identity runs through the lecture:

$$\log \frac{dp^*}{dp^u} = \int_0^1 \left(\partial_t r_t + b_t \cdot \nabla r_t + \frac{\sigma_t^2}{2} (\Delta r_t + \|\nabla r_t\|^2) - \frac{1}{2} \|u_t - U_t\|^2 \right) dt + \int_0^1 (U_t - u_t)^\top dW_t - \log Z,$$

weighted (§2), used as an importance weight for training (§3), or minimized in u (§4).

Where it stops Section 6 is the exception. The target is given by an energy, there is no pretrained model to tilt, and no Radon–Nikodym derivative is carried at all: the bridge matching sampler simulates its own endpoint, reads $-\nabla E$ there, and regresses. Two least squares, no weight — and the target enters as a fixed point of the loop rather than as a change of measure.

References

- Tara Akhound-Sadegh, Jarrod Rector-Brooks, Avishek Joey Bose, Sarthak Mittal, Pablo Lemos, Cheng-Hao Liu, Marcin Sendera, Siamak Ravanbakhsh, Gauthier Gidel, Yoshua Bengio, Nikolay Malkin, and Alexander Tong. Iterated denoising energy matching for sampling from Boltzmann densities. In *International Conference on Machine Learning (ICML)*, 2024. iDEM; arXiv:2402.06121.
- Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *Journal of Machine Learning Research*, 2025. arXiv:2303.08797.
- Luigi Ambrosio, Nicola Gigli, and Giuseppe Savaré. *Gradient Flows in Metric Spaces and in the Space of Probability Measures*. Lectures in Mathematics ETH Zürich. Birkhäuser, 2nd edition, 2008.
- Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3): 313–326, 1982.
- Denis Blessing, Mouyang Cheng, Maximilian Schebek, Jutta Rogal, Mingda Li, Carles Domingo-Enrich, and Yuanqi Du. JANUS: A multi-modal foundation neural sampler for disordered materials. *arXiv preprint arXiv:2608.19116*, 2026a.
- Denis Blessing, Lorenz Richter, Julius Berner, Egor Malitskiy, and Gerhard Neumann. Bridge matching sampler: Scalable sampling via generalized fixed-point diffusion matching. In *International Conference on Machine Learning (ICML)*, 2026b. BMS; arXiv:2603.00530.

- Benjamin Boys, Mark Girolami, Jakiw Pidstrigach, Sebastian Reich, Alan Mosca, and O. Deniz Akyildiz. Tweedie moment projected diffusions for inverse problems. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2310.06721.
- Gerard Brunick and Steven Shreve. Mimicking an itô process by a solution of a stochastic differential equation. *Annals of Applied Probability*, 23(4):1584–1628, 2013.
- Junhua Chen, Lorenz Richter, Julius Berner, Denis Blessing, Gerhard Neumann, and Anima Anandkumar. Sequential controlled Langevin diffusions. In *International Conference on Learning Representations (ICLR)*, 2025. SCLD; arXiv:2412.07081.
- Hyungjin Chung, Jeongsol Kim, Michael T. Mccann, Marc L. Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023. DPS; arXiv:2209.14687.
- Valentin De Bortoli, Michael Hutchinson, Peter Wirsberger, and Arnaud Doucet. Target score matching. *arXiv preprint arXiv:2402.08667*, 2024.
- Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential Monte Carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006.
- Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. Classifier guidance; arXiv:2105.05233.
- Carles Domingo-Enrich, Michal Drozdal, Brian Karrer, and Ricky T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.08861.
- Bradley Efron. Tweedie’s formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.
- Walter R. Gilks and Carlo Berzuini. Following a moving target — Monte Carlo inference for dynamic Bayesian models. *Journal of the Royal Statistical Society: Series B*, 63(1):127–146, 2001.
- Wei Guo, Jaemoo Choi, Yuchen Zhu, Molei Tao, and Yongxin Chen. Proximal diffusion neural sampler. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.03824.
- István Gyöngy. Mimicking the one-dimensional marginal distributions of processes having an itô differential. *Probability Theory and Related Fields*, 71(4):501–516, 1986.
- Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.
- Bahjat Kwar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising diffusion restoration models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. DDRM; arXiv:2201.11793.
- Guan-Hong Liu, Jaemoo Choi, Yongxin Chen, Benjamin Kurt Miller, and Ricky T. Q. Chen. Adjoint Schrödinger bridge sampler. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.22565.
- Radford M. Neal. Annealed importance sampling. *Statistics and Computing*, 11(2):125–139, 2001.
- Xinyu Peng, Ziyang Zheng, Wenrui Dai, Nuoqian Xiao, Chenglin Li, Junni Zou, and Hongkai Xiong. Improving diffusion models for inverse problems using optimal posterior covariance. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02149.
- Angus Phillips, Hai-Dang Dau, Michael John Hutchinson, Valentin De Bortoli, George Deligiannidis, and Arnaud Doucet. Particle denoising diffusion sampler. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.06320.
- Gareth O. Roberts and Richard L. Tweedie. Exponential convergence of Langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996.
- Raghav Singhal, Zachary Horvitz, Ryan Teehan, Mengye Ren, Zhou Yu, Kathleen McKeown, and Rajesh Ranganath. A general framework for inference-time scaling and steering of diffusion models. *arXiv preprint arXiv:2501.06848*, 2025. Feynman–Kac steering.
- Marta Skreta, Lazar Atanackovic, Avishek Joey Bose, Alexander Tong, and Kirill Neklyudov. Feynman–Kac correctors in diffusion: Annealing, guidance, and product of experts. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.02819.

- Jiaming Song, Arash Vahdat, Morteza Mardani, and Jan Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023. IIGDM.
- Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2011.13456.
- Dario Trevisan. Well-posedness of multidimensional diffusion processes with weakly differentiable coefficients. *Electronic Journal of Probability*, 21:1–41, 2016.
- Yinhuai Wang, Jiwen Yu, and Jian Zhang. Zero-shot image restoration using denoising diffusion null-space model. In *International Conference on Learning Representations (ICLR)*, 2023. DDNM; arXiv:2212.00490.
- Luhuan Wu, Brian L. Trippe, Christian A. Naesseth, David M. Blei, and John P. Cunningham. Practical and asymptotically exact conditional sampling in diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Twisted sequential Monte Carlo; arXiv:2306.17775.